KlaimSwift

FULL SYSTEM BUILD PROMPT

Kenya Insurance Claims Automation Platform

For: Antigravity AI Coding Agent | Solo Project | All 5 Phases | Firebase Backend

This document is a complete, fully-detailed build specification for the KlaimSwift insurance claims platform. It covers all five development phases and must be followed exactly as written. Every database schema, API integration, bot conversation flow, admin dashboard page, and security requirement is specified below.

Attribute	Value
Project Name	KlaimSwift — Kenya Insurance Claims Platform
Build Type	Solo Project — Full System (All 5 Phases)
Backend	Firebase (Firestore + Realtime DB + Storage + Auth)
Customer Interface	WhatsApp ONLY — No customer web portal
Admin Interface	Web Dashboard (HTML/CSS/JS)
Target Market	Kenya — KES currency, +254 phones, M-Pesa payments
Competition	PDC 2025/2026 — Actuaries Beyond Numbers

Phase 1 — Backend + WhatsApp Bot

1.1 Node.js / Express Server Routes

Build an Express server. All routes below are mandatory:

POST /webhook/whatsapp

- Receives all inbound WhatsApp messages from Meta
- Verifies webhook signature using X-Hub-Signature-256
- Parses: text, image, document, interactive, button reply
- Calls botEngine.process(phone, message) — async, respond 200 immediately

GET /webhook/whatsapp

- Meta webhook verification — checks hub.verify_token against env var WHATSAPP_VERIFY_TOKEN

- **POST /webhook/mpesa/result**
  - Receives Daraja B2C payment results
  - Updates mpesa_transactions and claims collections
  - Sends WhatsApp payout confirmation to customer on success

- **POST /api/admin/approve-claim — PROTECTED**
  - Triggers M-Pesa payout via Daraja API
  - Updates claim status → approved
  - Sends WhatsApp template to customer
  - Writes blockchain audit event

- **POST /api/admin/reject-claim — PROTECTED**
  - Updates claim status → rejected
  - Sends WhatsApp rejection message with reason

1.2 WhatsApp Bot State Machine

Every customer session persists in Firestore conversations/{phone}. All states below are mandatory:

State	Trigger	Bot Action	Next State
verify	Customer messages KlaimSwift	Ask for Policy Number	verify (loop) or menu
	Policy found	Greet by name, show main menu	menu
	Policy not found	Show 'not registered' + klaimswift.co.ke	verify
menu	Verified customer	Show: File Claim, Track, Policy, Payout, Agent	—
file-type	📝 File a Claim	Ask claim type (Motor/Medical/Property/Life)	file-date
file-date	Type selected	Ask incident date	file-amount
file-amount	Date entered	Ask estimated KES amount	file-desc
file-desc	Amount entered	Ask for description	file-docs
file-docs	Description saved	Ask for documents (photos, PDFs)	file-confirm
file-confirm	Docs processed	Show full claim summary card	—
	✅ Confirm	Create claim in Firestore, write blockchain, send claim ID	menu
	❌ Cancel	Clear wip, return to menu	menu
tracking	📋 Track Claim	Fetch all claims, render each as status card	menu
agent	📞 Talk to Agent	Set conversation as unassigned in admin inbox	agent

CRITICAL: Bot state must persist in Firestore. If a customer abandons mid-claim and returns 3 days later, the bot resumes exactly where they left off.

1.3 WhatsApp API Service (whatsappService.js)

Use Meta Cloud API directly. Base URL:
https://graph.facebook.com/v18.0/{PHONE_NUMBER_ID}/messages

Build these functions:

- sendText(to, body)
- sendTemplate(to, templateName, components)
- sendInteractiveButtons(to, body, buttons) — max 3 buttons
- sendInteractiveList(to, body, sections) — for menus > 3 options
- sendDocument(to, documentUrl, filename, caption)
- sendImage(to, imageUrl, caption)
- markAsRead(messageId)

CRITICAL: After 24 hours of customer inactivity, only approved WhatsApp templates can be sent. Bot engine must check lastWhatsappAt and switch to templates outside the 24-hour window.

1.4 WhatsApp Message Templates

Register all four templates in Meta Business Manager:

▸ **claim_submitted**

```
Your claim {{1}} has been submitted successfully.
Amount: KES {{2}}
Est. settlement: 2-3 business days. We'll update you at every step.
```

▸ **claim_approved**

```
Great news, {{1}}! Claim {{2}} has been APPROVED.
KES {{3}} will be sent to your M-Pesa ({{4}}) within minutes.
```

▸ **claim_rejected**

```
We're sorry, {{1}}. Claim {{2}} could not be approved.
Reason: {{3}}
For help: 0800 724 724 or reply to this message.
```

▸ **mpesa_sent**

```
✅ M-Pesa Sent! KES {{1}} → {{2}}. Confirmation: {{3}}. Claim: {{4}}
```

Phase 2 — AI Document Verification

2.1 Google Document AI Integration

Use Document AI Form Parser processor. For every document received via WhatsApp:

- Download from Meta media URL
- Upload to Firebase Storage
- Send to Google Document AI
- Extract: document type, dates, amounts, names, ID numbers
- Cross-reference extracted data against claim wip data
- Apply all 7 fraud detection rules (see below)
- Run async — never block WhatsApp conversation waiting for AI result
- Send 'processing...' message, then send AI result as separate message when ready

7 Mandatory Fraud Detection Rules

Rule	Trigger Condition	Action
Duplicate Claim	Same member + same incident date + same type within 30 days	Flag: SUSPECT
Amount Inflation	Repair estimate >200% above market rate for vehicle type	Flag: WARNING
Document Metadata	PDF creation date is AFTER the incident date	Flag: SUSPECT
Font Inconsistency	Different fonts detected on the same form	Flag: SUSPECT
Name Mismatch	Document name ≠ policy holder name	Flag: WARNING
GPS Inconsistency	Photo GPS metadata ≠ claimed incident location	Flag: WARNING
Velocity Check	>3 claims filed within any 12-month period	Route to adjuster queue

AI Confidence Score Thresholds

Score Range	Meaning	System Action
75 – 100	Low fraud risk	Auto-advance to payment queue — no human review needed
50 – 74	Medium risk	Route to adjuster queue for manual review
0 – 49	High fraud risk	Auto-flag as fraud alert, freeze claim, notify admin

---

Phase 3 — Admin Web Dashboard

Build a single-page HTML/CSS/JS admin dashboard. Connects to Firebase via JS SDK (CDN). Admin must be logged in via Firebase Auth. All pages use Firestore onSnapshot listeners — NO polling.

Design Specifications

Property	Value
Background	#111B21
Surface	#202C33
Surface2	#2A3942
Accent (green)	#25D366 (WhatsApp green)
Heading font	Syne (Google Fonts)
Body font	DM Sans (Google Fonts)
Charts	Chart.js (CDN)
Auth	Firebase Auth (email/password)

Dashboard Pages (10 Total)

Page	Key Content
1. Dashboard	Live KPIs: Claims Today, Pending, Fraud Flagged, Avg Settlement, M-Pesa Today. WhatsApp activity feed. New members feed. Recent claims table.
2. Claims Inbox	Full claims table with filters, search, approve/reject buttons, claim detail drawer with timeline.
3. WhatsApp Inbox	3-panel: conversation list | full chat thread | context sidebar with linked claim + AI results.
4. AI Document Verification	Verification queue, result badges (Authentic/Suspect/Warning), detection method accuracy bars.
5. Fraud Alerts	Flagged claims with AI signals. Actions: Confirm Fraud | Escalate | Dismiss.
6. Adjuster Queue	Claims with AI score 50–74. Inline approve/reject with notes.
7. Members Registry	Full table with live Firestore sync, search, WhatsApp message button, new member registration.
8. Analytics	6 Chart.js charts: volume, channel split, claim type, AI score distribution, settlement time, fraud rate.
9. M-Pesa Payouts	Full transaction table, retry failed payments, export CSV, Daraja status card.
10. IRA Compliance	Automated report downloads (PDF), DPA 2019 checklist, submission deadline tracker.

Phase 4 — M-Pesa Integration

4.1 Safaricom Daraja B2C Payout Flow

Build mpesaService.js. Full payout flow:

- Step 1: Get OAuth token via POST to /oauth/v1/generate (Basic auth)
- Step 2: Initiate B2C via POST to /mpesa/b2c/v1/paymentrequest

- Step 3: Handle result callback at /webhook/mpesa/result
- Step 4: Handle timeout at /webhook/mpesa/timeout
- Step 5 (success): Update claim.status = paid, create mpesa_transactions record, send WhatsApp confirmation
- Step 5 (failure): Mark transaction failed, notify admin, do NOT send customer false confirmation

> *CRITICAL: Never mark a claim as paid before Daraja confirms. Always wait for the callback.*

4.2 Admin Approval Trigger

When admin clicks Approve on a claim in the dashboard, this exact sequence runs:

- Update claim.status = approved in Firestore
- Write timeline event: { event: Approved, actor: admin.name, timestamp }
- Call mpesaService.initiateB2CPayout()
- Create mpesa_transactions record with status = pending
- Update claim.status = payout_initiated
- Send WhatsApp template claim_approved to customer
- On Daraja callback success: Update to paid, write blockchain event, send mpesa_sent template
- On Daraja callback failure: Revert to approved, push admin notification via Realtime DB

Phase 5 — Blockchain Audit Trail + IRA Compliance

5.1 Hyperledger Fabric Integration

Use Hyperledger Fabric SDK for Node.js. Single-channel permissioned network.

Network Configuration

- 1 organization: KlaimSwift
- 2 peers (for redundancy)
- 1 orderer
- Channel name: claims-channel

Build blockchainService.js with these functions

- writeAuditEvent({ eventType, claimId, actorId, actorType, data, timestamp })
- getClaimAuditTrail(claimId) — returns full immutable event list
- verifyEvent(eventHash) — confirms event has not been tampered

Write to blockchain for ALL of these events

- Claim filed (includes claim data hash)
- Each document uploaded (includes document hash)

- AI verification result (includes AI model version)
- Adjuster review (includes adjuster ID + decision)
- Claim approved (includes approver ID + timestamp)
- Claim rejected (includes reason hash)
- M-Pesa payout initiated (includes amount + recipient hash)
- M-Pesa payout confirmed (includes Safaricom TX ID)
- Fraud flag raised / resolved

Fallback: If full Hyperledger Fabric is too complex for solo deployment, implement SHA-256 hash chaining in Firestore as a fallback. Document this clearly and provide full Fabric setup guide for later.

5.2 Kenya Data Protection Act 2019 Compliance

Requirement	Implementation
AES-256 encryption	All PII encrypted at application layer before any Firestore write
PII fields to encrypt	name, phone, nationalId, mpesaPhone
Consent recording	Record whatsappOptIn=true + timestamp at first WhatsApp message
Consent withdrawal	If customer sends STOP: set whatsappOptIn=false, cease all outbound
Data export	GET /api/member/data-export returns all data for a member
Data deletion	DELETE /api/member/data-delete anonymizes member (subject to 7yr IRA retention)
Data residency	Firebase project must use africa-south1 (Cape Town) region — mandatory

---

Firebase Data Schema

Collection: members

Field	Type	Notes
memberId	string	Auto-generated
name	string	Encrypted — as on National ID
phone	string	Encrypted — +254XXXXXXXXX format
nationalId	string	Encrypted
policyNumber	string	Format: KEN-YYYY-{TYPE}-{6digits}
insuranceType	string	Motor Vehicle | Medical | Property | Life Insurance
status	string	active | suspended | expired
registeredAt	timestamp	
registeredVia	string	website | admin

Field	Type	Notes
claimsCount	number	
mpesaPhone	string	Encrypted — may differ from WhatsApp number
whatsappOptIn	boolean	
lastWhatsappAt	timestamp	Used to check 24hr message window

Collection: claims

Field	Type	Notes
claimId	string	Format: CLM-YYYY-KE-{7digits}
memberId	string	Reference to members
policyNumber	string	Denormalized
type	string	Motor Vehicle | Medical | Property | Life Insurance
incidentDate	string	
amountKES	number	Integer, no decimals
status	string	pending | ai_review | adjuster_review | approved | rejected | paid
channel	string	WhatsApp | Admin Portal
aiScore	number	0–100, higher = safer (less fraud risk)
aiVerdict	string	safe | review | flag
documents	array	[{ url, name, size, aiResult, aiReason, uploadedAt }]
blockchainTxHash	string	Returned from Hyperledger Fabric
timeline	array	[{ event, timestamp, actor, note }]
mpesaTransactionId	string	Safaricom TX ID after payout
filedAt	timestamp	
updatedAt	timestamp	

Environment Variables (.env)

All variables are mandatory. None are optional.

Firebase

```
FIREBASE_PROJECT_ID=
FIREBASE_PRIVATE_KEY=
FIREBASE_CLIENT_EMAIL=
FIREBASE_DATABASE_URL=
FIREBASE_STORAGE_BUCKET=
```

WhatsApp Business API

```
WHATSAPP_TOKEN=
WHATSAPP_PHONE_ID=
WHATSAPP_VERIFY_TOKEN=
WHATSAPP_BUSINESS_ACCOUNT_ID=
```

Google Document AI

```
GOOGLE_CLOUD_PROJECT_ID=
GOOGLE_DOCUMENT_AI_PROCESSOR_ID=
GOOGLE_APPLICATION_CREDENTIALS=./google-credentials.json
```

Safaricom Daraja API

```
MPESA_CONSUMER_KEY=
MPESA_CONSUMER_SECRET=
MPESA_SHORTCODE=
MPESA_INITIATOR_NAME=
MPESA_INITIATOR_PASSWORD=
MPESA_RESULT_URL=https://yourdomain.com/webhook/mpesa/result
MPESA_TIMEOUT_URL=https://yourdomain.com/webhook/mpesa/timeout
MPESA_ENV=sandbox
```

App

```
PORT=3000
NODE_ENV=development
ENCRYPTION_KEY=   # 32-byte hex key for AES-256
ADMIN_JWT_SECRET=
```

Project File Structure

```
klaimswift/
├── server.js                    # Express app entry point
├── .env
├── package.json
├── firebase-config.js           # Firebase Admin SDK init
├── routes/
│   ├── webhook.whatsapp.js      # WhatsApp webhook routes
│   ├── webhook.mpesa.js         # M-Pesa callback routes
│   └── api.admin.js             # Admin API routes (protected)
├── services/
│   ├── botEngine.js             # WhatsApp bot state machine
│   ├── whatsappService.js       # Meta Cloud API calls
│   ├── documentAI.js            # Google Document AI
│   ├── mpesaService.js          # Safaricom Daraja API
│   ├── blockchainService.js     # Hyperledger Fabric
```

```
│   ├── complianceService.js      # IRA report generators
│   └── encryptionService.js      # AES-256 PII encryption
├── admin/
│   ├── index.html                # Single-page admin dashboard (10 pages)
│   ├── login.html                # Firebase Auth login
│   └── assets/
└── docs/
    ├── SETUP.md                  # Full setup instructions
    ├── WHATSAPP_TEMPLATES.md     # Meta template registration guide
    ├── MPESA_SETUP.md            # Daraja API setup guide
    └── IRA_COMPLIANCE.md
```

Critical Rules for Antigravity

These rules are non-negotiable. Every rule below must be followed exactly:

#	Rule
1	Never build a customer web portal. The customer interface is WhatsApp ONLY.
2	All monetary amounts in KES (Kenyan Shillings). Never use USD, EUR, or generic currency.
3	Policy number format: KEN-YYYY-{TYPE}-{6digits}. TYPE = MTR, MED, PRO, or LIF.
4	Claim ID format: CLM-YYYY-KE-{7digits}. No exceptions.
5	Bot state must persist in Firestore. Customer can abandon and resume days later.
6	Admin dashboard must use onSnapshot listeners. Never use polling or manual refresh.
7	Blockchain records are append-only. Never update or delete any blockchain event.
8	Never mark a claim as 'paid' before receiving Daraja callback confirmation.
9	Document AI runs async. Never block WhatsApp conversation waiting for AI result.
10	Firebase region must be africa-south1 (Cape Town). Required by Kenya DPA 2019.
11	All admin routes require Firebase Auth. No unauthenticated access to any admin function.
12	Implement retry logic (exponential backoff) for failed WhatsApp message sends.
13	All PII fields must be AES-256 encrypted before any Firestore write operation.

Deliverables Checklist

Antigravity must produce ALL of the following files:

- server.js — Complete Express server with all routes
- services/botEngine.js — Full state machine, all states, Firestore persistence
- services/whatsappService.js — All message types, direct Meta Cloud API
- services/documentAI.js — Google Document AI + all 7 fraud rules

- services/mpesaService.js — Full Daraja B2C flow including callbacks
- services/blockchainService.js — Hyperledger Fabric write/read/verify
- services/complianceService.js — IRA report generators (PDF output)
- services/encryptionService.js — AES-256 for all PII fields
- admin/index.html — Single file, all 10 admin pages, live Firestore data
- admin/login.html — Firebase Auth login page
- Firebase security rules — Firestore + Storage
- Firestore indexes — firestore.indexes.json
- WhatsApp templates — all 4 templates, text formatted for Meta Business Manager
- .env.example — every variable documented
- docs/SETUP.md — step by step: Firebase, WhatsApp API, Daraja, Document AI, Railway/Render
- docs/WHATSAPP_TEMPLATES.md — exact guide to register templates in Meta
- tests/ — test files for bot engine, WhatsApp, M-Pesa, Document AI, blockchain

---

KlaimSwift — Build Prompt v1.0

Prepared for Antigravity | Solo Project | PDC 2025/2026 | Confidential